DIGITOPIA

Sirr: A Secure and Anonymous Reporting System

Building Trust Through Secure and Anonymous Communication

Philopater Guirgis

Yousif Abdulhafiz

Ahmed Lotfy

Ramy George

Patrick Ramez

Cybersecurity
Silent Vigilant: Digital Tools for a Safer Society
Anonymous Crime Reporting

30 September 2025

DIGITOPIA

Sirr: A Secure and Anonymous Reporting System

Philopater Guirgis (Team Leader)
National ID: 30511011606457
Frontend & Backend

Yousif Abdulhafiz
AI/ML & Backend

Ahmed Lotfy
AI/ML & Backend

Ramy George
Frontend & Backend

Patrick Ramez
Testing

Team Name: A7fad Tohot

Project Name: Sirr

Track Name: Cybersecurity
Challenge Name: Silent Vigilant: Digital Tools for a Safer Society
Sub-Challenge Name: Anonymous Crime Reporting

Digitopia: Cybersecurity Track
30 September 2025

Contents

List of Figures

List of Tables

1

INTRODUCTION

1.1 Problem Statement

Crime underreporting is a persistent global challenge that undermines public safety and the effectiveness of law enforcement. In Egypt, for instance, studies suggest that up to 99% of gender-based violence incidents remain unreported due to fear of retaliation, social stigma, and mistrust in institutions. Similar patterns are observed in cases of harassment, cybercrime, and other sensitive offenses, where victims and witnesses hesitate to speak out. Traditional reporting channels, such as hotlines or in-person complaints, often fail to guarantee anonymity, leaving individuals vulnerable and discouraged from seeking justice. This gap creates a silent crisis in which countless incidents go unnoticed, weakening both community security and institutional accountability.

1.2 Solution Overview

To address this pressing issue, we propose **Sirr: A Secure and Anonymous Reporting System**, a privacy-first platform designed under the Digitopia Cybersecurity Track. Sirr is a web-based solution that empowers citizens to report crimes safely, anonymously, and without fear of exposure. The platform integrates advanced cryptographic techniques, streamlined user experience, and AI-powered intelligence to ensure that every report is both secure and actionable. Unlike conventional approaches, Sirr does not merely promise anonymity—it provides verifiable guarantees through its transparent and auditable architecture.

1.3 Key Features Highlight

The system introduces several innovative features:

- **Tiered Anonymity:** Users may choose between standard protection for simple cases or enhanced anonymity for high-risk scenarios.

- **Secure Two-Way Communication:** Each report generates a unique, encrypted tip ID that allows investigators and reporters to exchange follow-up messages without revealing identities.
- **Privacy-Preserving Media Handling:** Client-side metadata scrubbing and end-to-end encryption ensure that uploaded evidence cannot leak sensitive information.
- **Zero-Log Policy:** No personally identifiable information (PII) or IP addresses are stored, reinforcing trust in the system.
- **AI-Assisted Analysis:** Integrated natural language processing models filter spam, flag false submissions, and classify urgency levels to prioritize critical reports.

1.4 Target Impact

Sirr aims to rebuild trust between communities and institutions by creating a reliable channel for anonymous crime reporting. By lowering the barriers to reporting and safeguarding user privacy, the platform can significantly increase the number of incidents reported, particularly in sensitive cases like gender-based violence, corruption, and harassment. On the institutional side, Sirr delivers high-quality, structured intelligence that enables investigators to act swiftly and effectively. Ultimately, the project contributes to a safer society by bridging the gap between silent victims and responsive authorities.

1.5 Project Objectives

The objectives of the project are as follows:

1. Develop a web-based application that enables secure, anonymous, and user-friendly reporting of crimes.
2. Integrate end-to-end cryptographic mechanisms to guarantee reporter privacy and data confidentiality.
3. Provide investigators with a secure portal for triage, evidence management, and follow-up communication.
4. Incorporate AI tools to detect spam submissions and assess the urgency of legitimate reports.
5. Ensure transparency, accountability, and public trust by adopting open-source and auditable components.

1.6 Target Users and Stakeholders

Sirr is designed with multiple user groups in mind:

- **Concerned Citizens:** Individuals who may not be technologically advanced but require a simple and safe way to report incidents.
- **Witnesses:** People who observe crimes or threats and need a quick, discreet method to share evidence.
- **Victims of Sensitive Crimes:** Survivors of gender-based violence, harassment, or corruption who prioritize anonymity and protection.
- **Investigators and Authorities:** Law enforcement and authorized personnel who receive structured, prioritized reports for follow-up.
- **Community Stakeholders:** NGOs, advocacy groups, and policymakers who benefit from increased reporting data to drive reforms and awareness campaigns.

In summary, Sirr is not just a technical project but a social innovation aimed at empowering communities through technology. By combining usability, security, and intelligence, the platform aspires to transform crime reporting from a fearful, stigmatized act into a safe and trusted civic duty.

2

Platform Overview

2.1 What is SIRR?

Sirr: A Secure and Anonymous Reporting System is a web-based platform designed to provide citizens with a trusted, privacy-first channel for reporting crimes and safety concerns. It functions as a bridge between community members and investigative authorities, ensuring that sensitive information can be shared without exposing the reporter's identity. Unlike conventional hotlines or reporting apps, Sirr integrates cryptographic guarantees, frictionless user experience, and AI-powered intelligence into a single ecosystem.

The system operates entirely within a secure architecture that emphasizes three design priorities: *anonymity, usability, and accountability*. By stripping metadata, encrypting submissions, and enforcing a zero-log policy, it ensures the reporter's digital footprint is minimized. At the same time, investigators benefit from structured, high-quality intelligence and a secure portal for case management. Thus, Sirr positions itself not only as a technical product but as a societal tool to enhance community trust in institutions.

2.2 Core Value Proposition

The strength of Sirr lies in its ability to combine ease-of-use with verifiable security guarantees. Its core value proposition is structured around three pillars:

- **Verifiable Anonymity:** Unlike traditional reporting channels that claim to preserve anonymity, Sirr provides cryptographic proof of confidentiality. Client-side encryption ensures that only investigators with the appropriate keys can decrypt report contents.
- **Frictionless Access:** The platform requires no accounts, downloads, or installations. Reports can be submitted through any modern web browser, making the system accessible even to low-tech users.
- **Actionable Intelligence:** Reports are enriched with structured data, optional media uploads, and a secure tip ID that enables anonymous follow-up

communication. An AI analysis layer further assists by filtering out spam and prioritizing urgent cases.

Together, these pillars establish Sirr as a platform that balances security with usability, creating both trust and effectiveness.

2.3 System Architecture Diagram

Sirr is built on a modular, layered architecture that separates responsibilities between client-side, server-side, and analysis components. Figure **??** illustrates the high-level system architecture.
The architecture consists of the following major components:

- **Reporter Client:** A web application responsible for user interaction, client-side encryption, and metadata scrubbing.
- **Submission Gateway:** A hardened backend API that receives encrypted reports while enforcing a strict no-log policy.
- **Messaging and Storage Layer:** RabbitMQ and PostgreSQL used to decouple submissions, add resilience, and securely store encrypted blobs.
- **Investigator Portal:** A secure dashboard with role-based access control and two-factor authentication for authorized investigators.
- **AI Analysis Engine:** NLP-based modules that detect spam, classify urgency, and provide reasoning to support investigator decisions.
- **Key Management Service (KMS):** Responsible for public/private key handling, ensuring that decryption keys are tightly controlled.

2.4 Key Features at a Glance

Table 2.1 summarizes Sirr's key features across usability, security, and intelligence dimensions.
This combination of features ensures that Sirr is not only secure but also practical and scalable, setting it apart from both traditional channels and complex, high-barrier anonymous reporting systems.

Category	Feature Description
Anonymity	Zero-log policy for IP/PII; client-side encryption and metadata scrubbing ensure verifiable anonymity.
Accessibility	Web-based, no account creation or app installation; simple guided flow supports low-tech users.
Communication	Encrypted tip IDs enable secure two-way messaging without revealing identity.
Media Handling	Privacy-preserving uploads with automatic metadata removal before transmission.
Investigator Tools	Role-based portal with two-factor authentication, audit logs, and secure evidence management.
AI Assistance	Automated spam detection and urgency classification, with human-readable reasoning for transparency.
Transparency	Open-source components and documentation enable public auditing and verification.

Table 2.1: *Key features of the Sirr platform.*

3

User Roles & Workflows

3.1 Reporter Experience

3.1.1 Anonymous Submission Process

The system is designed from the ground up to guarantee reporter anonymity. At no point during submission is personally identifiable information (PII) required or logged. All sensitive data is encrypted directly in the reporter's browser **before** being transmitted to the backend.

Core Anonymity Features

- **End-to-End Encryption (E2EE):** Report details and attachments are encrypted locally using a hybrid public-key encryption scheme (X25519-XChaCha20-Poly1305). Only authorized personnel with the correct private keys can decrypt the submissions.
- **No User Accounts:** Reporters do not need accounts or email addresses, eliminating common sources of identity linkage.
- **Ephemeral Keys:** Each submission uses a temporary one-time key pair. The private key never leaves the reporter's device.
- **No IP Logging:** Middleware ensures that IP addresses and request headers are not stored.
- **Secure Access Key:** After submission, reporters receive a unique random `access_key`, which is the only way to follow up on the report. It is never tied to an identity.

Anonymous Submission Flow

Figure 3.1 shows the step-by-step process of anonymous submission.

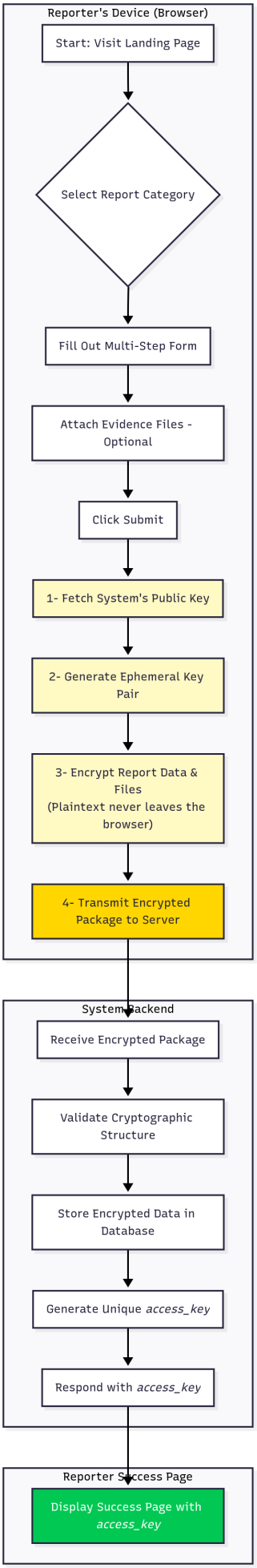

Figure 3.1: *Anonymous submission flow from reporter device to backend.*

3.1.2 User Interface (UI) Overview

The UI is designed for clarity and reassurance, guiding reporters through each step. A structured classification system, informed by government and academic sources (FBI, DHS, EU, EPA, CAPC, UCLA), ensures reports are consistently categorized.

Primary UI Components

UI Component	Purpose
Landing Page	Entry point offering “Start New Report,” “Follow Up,” and emergency contact info.
Report Category Selection	Guides reporters from broad to specific categories to ensure accurate classification.
Report Submission Form	Multi-step form tailored to incident type, covering details, suspects, and evidence.
File Upload Interface	Secure drag-and-drop for encrypted attachments (images, videos, documents).
Submission Success Page	Displays unique `access_key` and instructions to save it securely.
Follow-Up Modal	Allows reporters to check status using their `access_key`.
Anonymity Guide	Provides safe-reporting best practices to enhance user protection.

Table 3.1: *Key user interface components for reporters.*

UI Navigation Flow

Figure 3.2 illustrates how reporters move between the main UI screens.

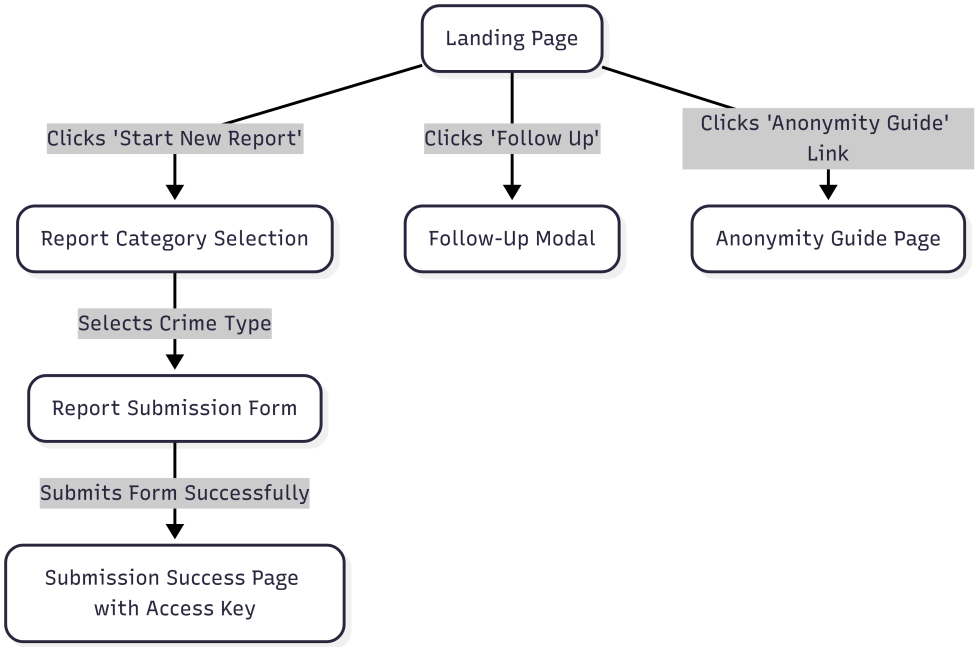

Figure 3.2: *Navigation flow between reporter UI components.*

3.1.3 Privacy and Security Measures

The system employs a layered security model to protect data and anonymity:

1. **Key Exchange:** The browser fetches the system's public key.
2. **Local Encryption:** An ephemeral key pair is generated on-device; data is encrypted with symmetric keys.
3. **Key Wrapping:** Symmetric keys are encrypted using the ephemeral private key and system's public key.
4. **Transmission and Storage:** Only encrypted data and keys are sent; no plaintext is stored.
5. **Decryption:** Only authorized backend processes can decrypt using the private key stored in a secure environment.

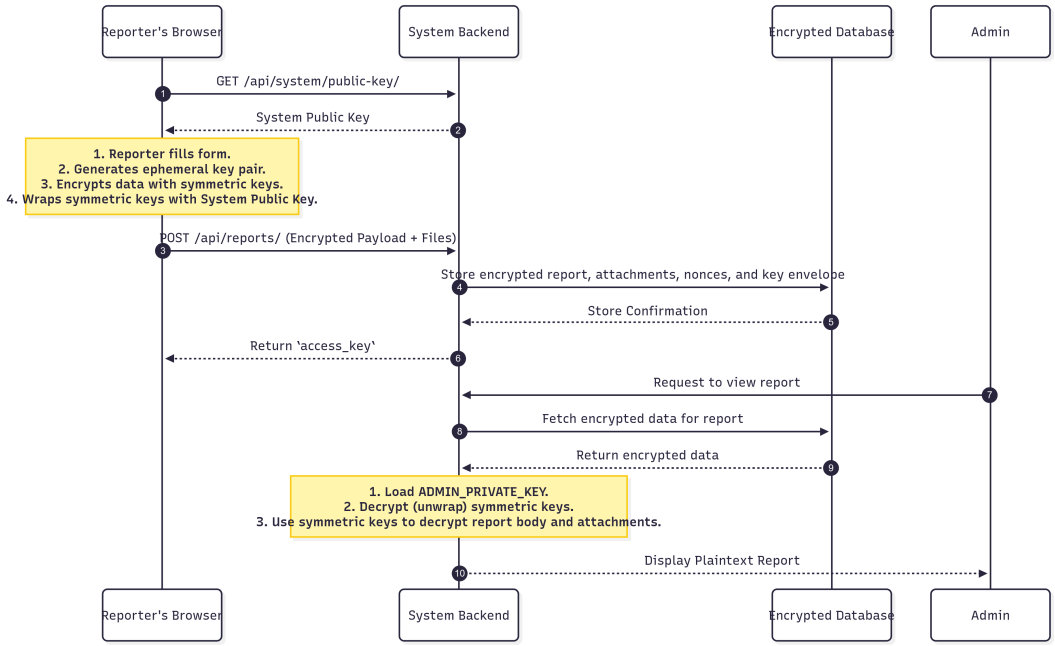

Figure 3.3: *Security architecture for encrypted reporter submissions.*

3.1.4 Safety Guidelines for Reporters

Reporters are advised to combine technical safeguards with safe practices:

- Use personal or public devices, not work computers, when submitting reports.
- Prefer secure connections with Cloudflare WARP or reputable no-log VPNs.
- Use privacy-focused browsers (e.g., Brave, Firefox Private Mode).
- Enable DNS-over-HTTPS (DoH) in browser settings.
- Avoid including names, phone numbers, or identifiers unless absolutely necessary.

These guidelines, combined with the platform's technical safeguards, ensure reporters can confidently share critical information while maintaining full anonymity.

3.2 Admin Dashboard

The admin dashboard serves as the central management interface for system administrators to oversee report submissions, manage user accounts, perform cryptographic operations, and trigger AI-powered analysis. Built on Django's admin framework, it provides a secure, role-based interface accessible at `/admin/`.

3.2.1 Access Control and Core Features

The system implements role-based access control with two primary roles: **Superusers** have unrestricted access to all reports and system configurations, while **Staff/Caseworkers** can only access reports explicitly assigned to them through custom queryset filtering.

Report Management

The report management interface provides:

- **Report Listing**: Filterable, searchable list view with metadata including ID, title, status, priority, and timestamps. Supports filtering by status (New, In Progress, Resolved, Closed) and priority levels.
- **On-Demand Decryption**: Administrators decrypt reports using the admin private key, with content displayed as formatted JSON. A collapsible section shows cryptographic metadata for audit purposes.
- **Report Assignment**: Inline interface for assigning reports to caseworkers. The system automatically re-encrypts the report's symmetric key for each assignee using their public key, enabling secure access without exposing the admin's private key.

User Management

Key capabilities include user creation with customizable permissions, an investigator invitation system with single-use time-bound tokens, security status monitoring (failed logins, account locks), and public key management for the re-encryption mechanism.

AI Analysis

Administrators can select multiple reports and trigger AI analysis through a custom admin action. The system decrypts reports and dispatches asynchronous Celery tasks for spam detection and urgency classification. Results include confidence scores, classifications, and AI-generated reasoning, with model version tracking for performance evaluation.

3.2.2 Administrative Workflows

Report Assignment Workflow

The secure delegation process: (1) Administrator selects a report and chooses caseworkers from a filtered dropdown showing only active users with valid public keys. (2) Upon saving, the system retrieves the admin private key, decrypts the report's symmetric key using the reporter's ephemeral public key, generates a new ephemeral key pair, and re-encrypts the symmetric key for each caseworker. (3) A checkmark indicator confirms successful key re-encryption. (4) Assigned caseworkers can now decrypt the report using their private key.

Investigator Onboarding Workflow

The secure account creation process: (1) Administrator creates an invitation with the investigator's email and role. (2) System generates an inactive user account and cryptographically secure, single-use token with configurable expiration. (3) Administrator securely delivers the onboarding URL out-of-band. (4) Investigator completes two-step activation: setting a password and uploading their client-generated public key bundle (Step 1), then verifying TOTP functionality to activate the account (Step 2). (5) Administrator verifies successful onboarding by checking account status and public key presence.

AI Analysis Workflow

The automated triage process: (1) Administrator selects reports and triggers the "Run AI Analysis" action. (2) System decrypts each report without existing analysis and dispatches Celery tasks. (3) Workers process tasks using the DSPy-based pipeline: removing redundant keys, converting to YAML format, executing spam detection and urgency classification modules, and generating reasoning explanations. (4) Results are stored with spam classification, confidence score, urgency level, and reasoning. (5) Administrators review results through the AI Analysis interface.

3.2.3 Authentication and Key Management

Admin Authentication

The multi-layered system includes: Django session-based authentication with Argon2 password hashing, optional TOTP-based 2FA with replay attack prevention, automatic account lockout after failed attempts, and Redis-backed session management with configurable timeouts.

Admin Private Key Storage

The admin private key enables decryption of all submitted reports. Key management differs between environments:

Development Environment The key is stored as an environment variable (`ADMIN_PRIVATE_KEY`) in the `.env` file. Generated during initial setup using the `create_admin_with_keys` command, which creates an X25519 key pair via PyNaCl, stores the public key in the database, outputs the base64-encoded private key for manual `.env` addition, and creates a placeholder Kyber KEM key for future post-quantum readiness.

Production Environment The key must be stored in a Key Management Service (KMS) providing: hardware security through HSMs with tamper-resistant protection, fine-grained access control policies, comprehensive audit logging of key access, secure key rotation capabilities, and encryption at rest with decryption only in the secure KMS environment. Implementation should integrate with cloud provider KMS (AWS KMS, Google Cloud KMS, Azure Key Vault) or enterprise infrastructure, retrieving the key at startup and caching securely in memory.

3.2.4 Security Considerations

The dashboard implements multiple security measures: comprehensive error handling for cryptographic operations to detect key mismatches and corruption, audit logging of critical operations (decryption, assignments, account modifications), input validation and sanitization, Django CSRF protection, permission verification for all views and actions, and secure key handling that never exposes private keys in logs or interfaces.

3.2.5 Technical Implementation

Built using Django's admin framework with extensive customization: specialized `ModelAdmin` subclasses for core models, tabular inline editing for report assignments, custom actions for bulk operations, additional views for specialized workflows, custom templates for enhanced UX, and queryset optimizations with prefetch and select_related. The implementation leverages Django ORM, PyNaCl for cryptography, and Celery for asynchronous processing.

3.3 Investigator Tools

3.3.1 Case Management Interface

Purpose and Main Features

The Case Management Interface is the primary workspace for investigators, providing access to assigned cases while enforcing strict access controls. It combines usability with security to ensure that sensitive information is only accessible by authorized users.

Main features:

- **Centralized Case Dashboard:** A grid view listing assigned cases with details such as status, labels, attachments, and submission dates.
- **Advanced Filtering and Search:** Investigators can search by Case ID or Label and filter by priority (e.g., "Only Important").
- **Inline Case Updates:** Quick actions such as marking cases as important or adding labels can be performed directly from the dashboard.
- **Secure Case Detail View:** A dedicated page with a structured, tab-based layout showing case details and evidence.
- **On-Demand Decryption:** Case details remain encrypted at rest and are only decrypted in the investigator's browser with their private key.

User Roles

The platform enforces role-based separation of duties:

- **Investigator (Caseworker):** Can only access cases explicitly assigned to them. Their cryptographic key bundle ensures that only they can decrypt case data.
- **Administrator (Superuser):** Has full system access, including case triage, user management, and assignments.

3.3.2 Case Lifecycle Workflows

Case Initiation

1. A public report is submitted through the system's API.
2. Encrypted report data is stored with a default status of *New*.
3. The Administrator reviews the case in the triage queue.

Case Triage and Updates

1. Administrator reviews new cases, optionally assisted by AI analysis for urgency and spam detection.
2. Case is assigned to an investigator.

3. When the case is first opened by the investigator, its status changes from *New* to *Opened*.
4. Investigators can update metadata (e.g., mark as important, add labels).

Case Closure

1. Investigator or Administrator marks the case as *Closed*.
2. Closed cases remain archived but are hidden from active queues.

3.3.3 Permissions and Access Control

The system enforces a layered security model:

- **Authentication:** Mandatory two-factor authentication (password + TOTP).
- **Authorization:** Backend API enforces strict checks, preventing access to unassigned cases.
- **Cryptographic Enforcement:** Only investigators with the correct private key can decrypt case data, even if API access is compromised.

3.3.4 System Diagrams

Entity-Relationship Diagram (ERD)

Figure 3.4 models the primary entities and their relationships in the case management process.

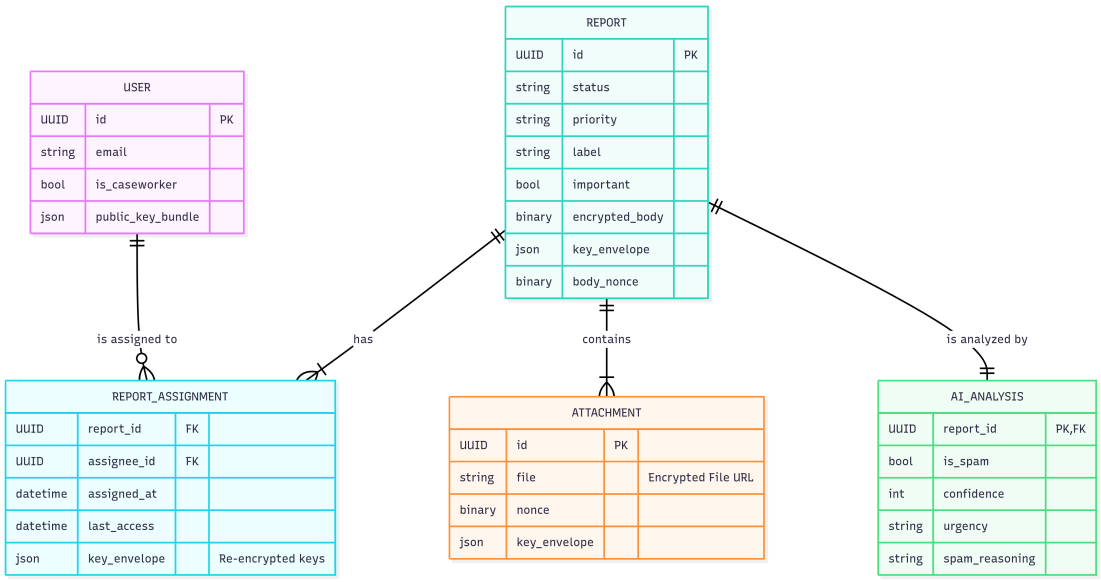

Figure 3.4: *Entity-Relationship Diagram (ERD) for investigator case management.*

3.3.5 Assignment and Tracking Workflows

Case Assignment Process

Cases are assigned to investigators by administrators through a cryptographically enforced process:

1. Administrator selects a case for assignment.
2. The server decrypts the report's symmetric keys using `ADMIN_PRIVATE_KEY`.
3. The keys are re-encrypted with the chosen investigator's public key.
4. A `ReportAssignment` record links the case to the investigator and stores the re-encrypted key bundle.
5. The case appears in the investigator's dashboard.

Tracking Mechanisms

- **Status Updates:** Case status transitions automatically (*New* → *Opened* → *Closed*).
- **Audit Trail:** The `last_access` timestamp records every investigator access.
- **Flags and Labels:** Important cases can be flagged, and labels can be added for tracking.

Escalation and Reassignment

- **Reassignment:** Administrators can reassign cases by generating new key envelopes for another investigator.
- **Escalation:** Currently manual, based on administrator review of pending cases.

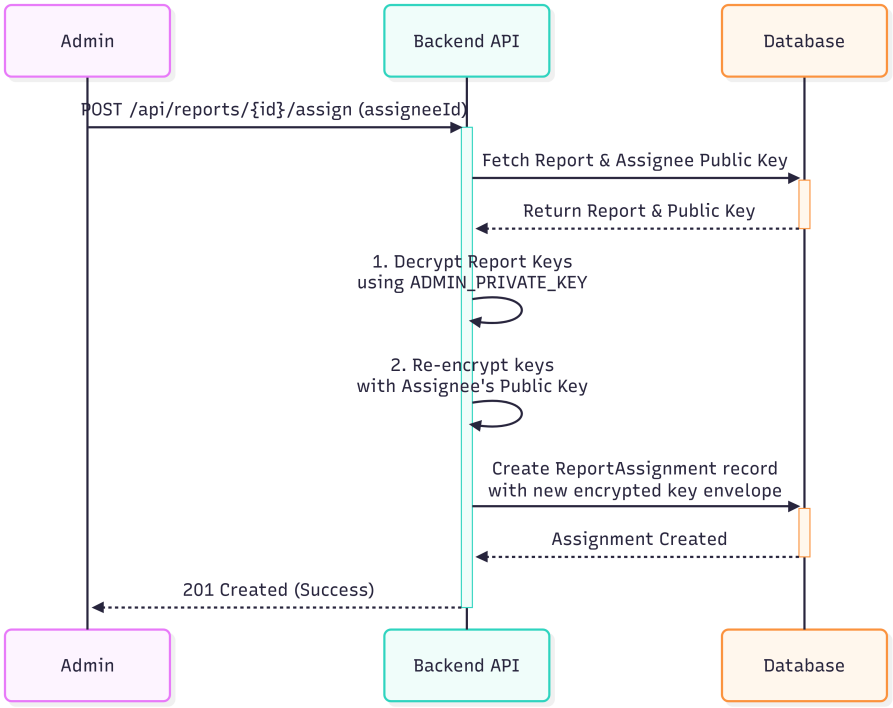

Figure 3.5: *Assignment workflow showing cryptographic key re-encryption.*

3.3.6 Investigation Lifecycle

Lifecycle Stages

1. **Initiation:** A new encrypted report is submitted and stored with status *New*.
2. **Triage and Assignment:** Administrator reviews and assigns the case.
3. **Evidence Analysis:** Investigator decrypts and reviews evidence (status: *Opened*).
4. **Reporting:** Investigator completes their analysis. Reporting features are external to the current system.
5. **Closure:** Investigator or Administrator marks case as *Closed*, moving it to archive.

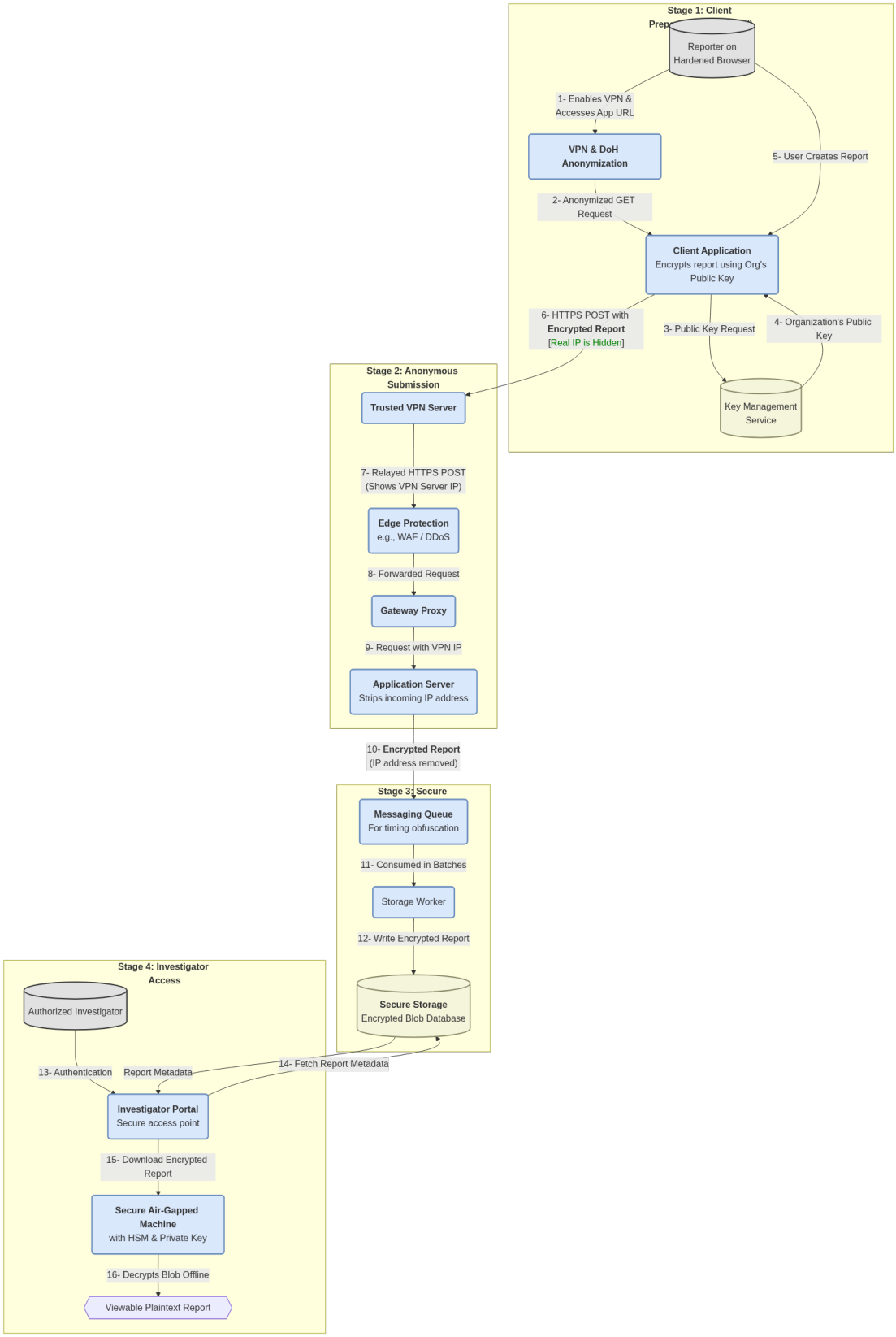

Figure 3.6: *Data Flow Diagram*

3.4 AI Analysis

3.4.1 Problem Statement

Our reporting system handles sensitive incident reports submitted by users through an end-to-end encrypted platform. Each report contains structured JSON data describing various types of incidents, ranging from cybercrime and violence to missing persons and public safety concerns. To effectively prioritize and manage these reports, investigators need automated assistance in two critical areas: identifying spam submissions and assessing the urgency level of legitimate reports.

Input Data

The AI analysis system processes:

- **Report Content**: Structured JSON data containing incident descriptions, timestamps, locations, and category-specific details submitted through dynamic forms

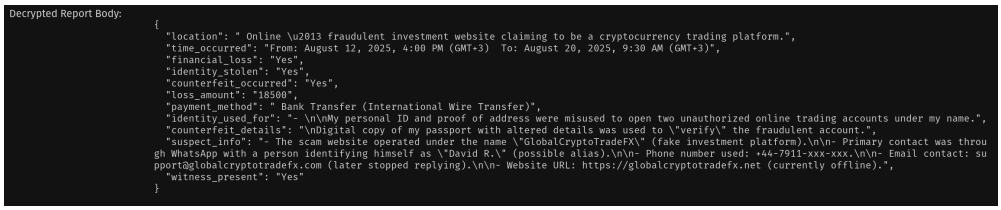

Figure 3.7: *Report Content*

- **Training Data**: Publicly available datasets containing labeled examples of spam and legitimate reports with urgency classifications (low, medium, high, critical).
  Crime Reports Dataset (Hugging Face)
- **Validation Data**: Separate dataset used to evaluate model performance and prevent overfitting

Expected Output

For each analyzed report, the system must produce:

- **Spam Classification**: Binary determination (spam/not spam) with confidence score (0-100%)
- **Urgency Level**: Four-tier classification (low, medium, high, critical) for criminal investigation prioritization
- **Reasoning**: Human-readable explanations for both classifications to support investigator decision-making

Critical Constraints

The implementation faces two fundamental constraints that significantly impact our architectural decisions:

1. **Privacy Requirements**: User data must remain confidential and cannot be transmitted to external cloud services. This necessitates the use of locally-hosted Large Language Models (LLMs) that run entirely within our infrastructure, ensuring that sensitive report content never leaves our controlled environment.
2. **Hardware Limitations**: Our available GPU resources are limited to consumer-grade hardware with modest VRAM capacity. This constraint eliminates the possibility of running large-scale models (e.g., 70B+ parameter models) and requires careful selection of smaller, optimized models that can deliver acceptable performance within our hardware envelope.

These constraints create a challenging optimization problem: we must achieve high accuracy in spam detection and urgency classification using only small, locally-hosted models running on limited hardware.

3.4.2 Solution

To address these constraints while maintaining high accuracy, we developed a solution combining modern prompt engineering frameworks with efficient local inference.

Technology Stack

Our implementation leverages:

- **DSPy Framework**: A declarative framework for programming language models that treats prompts as learnable parameters rather than hand-crafted strings. DSPy enables systematic optimization of prompts through automated techniques.
- **Ollama**: A lightweight inference engine optimized for running LLMs locally with GPU acceleration. Ollama provides efficient model serving with minimal overhead.
- **Gemma 3 4B Model**: Google's compact 4-billion parameter language model, specifically chosen for its balance between performance and resource requirements. The model fits comfortably within our GPU memory constraints while maintaining strong reasoning capabilities.
- **Celery Task Queue**: Asynchronous task processing system that handles AI analysis jobs in the background, preventing blocking operations and ensuring responsive user experience.

Architectural Approach

The system employs a modular architecture with two specialized analysis modules:

1. **Spam Detection Module**: Implements Chain-of-Thought reasoning to analyze report content and determine authenticity. The module outputs a binary classification with confidence score and detailed reasoning.

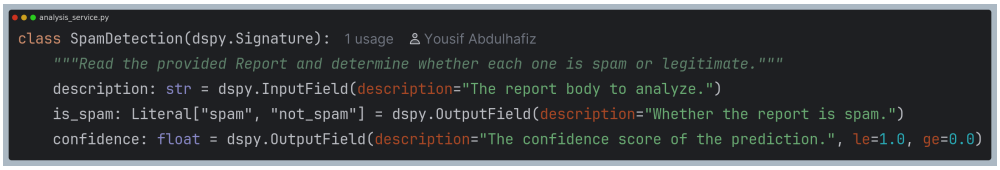

Figure 3.8: *Spam Module*

2. **Urgency Classification Module**: Evaluates the severity and time-sensitivity of reports to assign appropriate priority levels for investigator attention.

Figure 3.9: *Urgency Module*

Both modules operate independently but are orchestrated through a unified analyzer service that processes reports asynchronously. When a new report is submitted, the system triggers a background task that:

1. Extracts and cleans the report content
2. Converts structured data to YAML format for optimal LLM processing
3. Executes both analysis modules in parallel
4. Stores results in the database for investigator access

3.4.3 Optimization Pipeline

The key innovation in our approach is the systematic optimization of prompts and reasoning strategies through an automated pipeline.

1. **Data Preparation**: Training, validation, and test sets are loaded and preprocessed. Reports are cleaned to remove redundant metadata while preserving essential content.
2. **Baseline Evaluation**: The initial model with default prompts is evaluated against the validation set to establish baseline performance metrics.

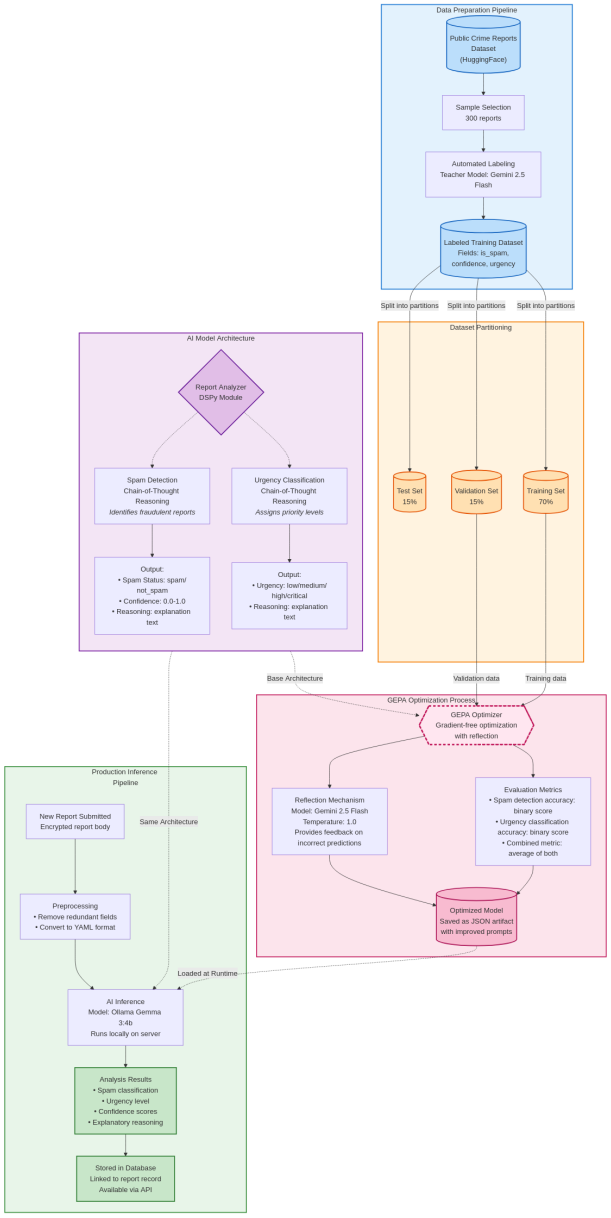

Figure 3.10: *AI Analysis Optimization Pipeline*

3. **GEPA Optimization**: The optimizer systematically explores prompt variations and reasoning strategies, using feedback from validation results to guide the search process.
4. **Model Compilation**: The best-performing configuration is compiled into an optimized program that can be deployed to production.
5. **Testing**: Final evaluation on held-out test data confirms generalization performance.

3.4.4 GEPA: Generalized Prompt Optimization

GEPA (Generalized Prompt Optimization with Adaptive Feedback) is an advanced prompt optimization technique that treats prompt engineering as a machine learn-

ing problem rather than a manual craft.

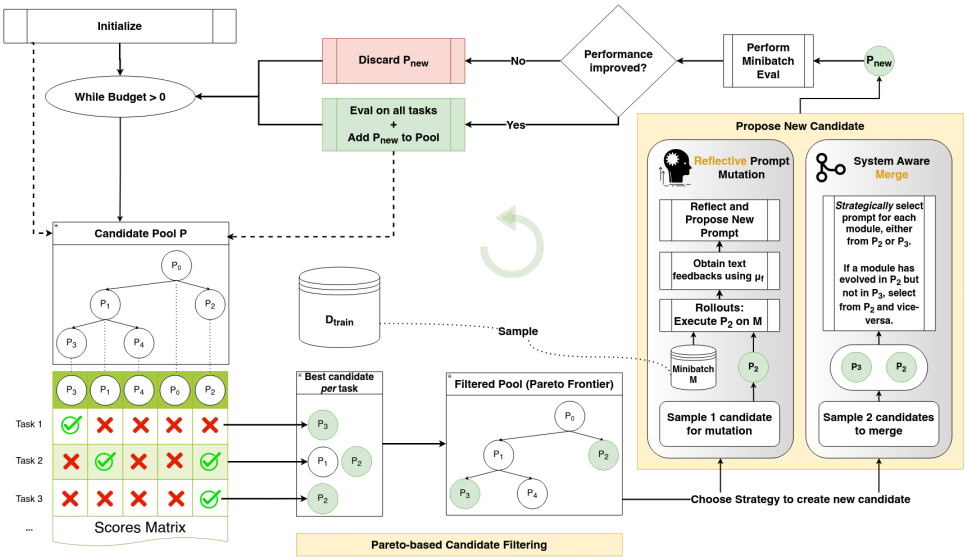

Figure 3.11: *GEPA Optimization Process: The system iteratively refines prompts through evaluation, reflection, and adaptation cycles, guided by performance feedback from validation data.*

Core Concept

Traditional prompt engineering relies on human intuition and trial-and-error to craft effective prompts. GEPA automates this process by:

- **Treating Prompts as Parameters**: Instead of fixed strings, prompts become learnable components that can be systematically optimized.
- **Using Feedback Loops**: The optimizer evaluates each prompt variation against labeled data and uses the results to guide further refinement.
- **Leveraging Meta-Learning**: A separate ”reflection” LLM (in our case, Google’s Gemini 2.5 Flash) analyzes failures and suggests improvements, creating a self-improving system.

Why GEPA?

We chose GEPA for several compelling reasons:

1. **Systematic Optimization**: Rather than manually testing hundreds of prompt variations, GEPA explores the prompt space intelligently, focusing on promising directions.
2. **Task-Specific Adaptation**: The optimizer tailors prompts specifically to our spam detection and urgency classification tasks, incorporating domain knowledge automatically.

3. **Feedback Integration**: By providing detailed feedback on incorrect predictions, GEPA helps the model learn from mistakes and refine its reasoning strategies.
4. **Resource Efficiency**: GEPA achieves strong performance improvements without requiring model fine-tuning or additional training data, making it ideal for our resource-constrained environment.

Implementation Details

Our GEPA configuration uses:

- **Metric Function**: Custom scoring that averages spam detection accuracy and urgency classification accuracy, with detailed feedback for each module
- **Reflection LLM**: Google Gemini 2.5 Flash with temperature 1.0 for creative prompt refinement
- **Optimization Mode**: Medium automation level, balancing exploration breadth with computational cost
- **Parallel Processing**: 4 threads for efficient evaluation of prompt candidates

3.4.5 Results

The optimization pipeline delivered substantial improvements in model performance, validating our approach to prompt engineering.

Performance Improvements

Through GEPA optimization, we achieved:

- **Baseline Accuracy**: Initial model with default prompts achieved approximately 30% accuracy on combined spam detection and urgency classification tasks
- **Optimized Accuracy**: After GEPA optimization, accuracy improved to approximately 80%, representing a 2.7x improvement
- **Reasoning Quality**: Optimized prompts produce more detailed and contextually relevant explanations, improving investigator trust in AI recommendations

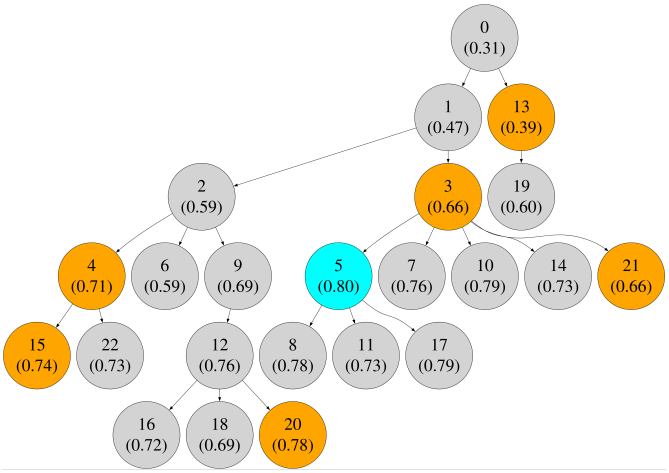

Figure 3.12: *AI Analysis Performance Improvement*

Optimized Prompts Used

To enable these improvements, the following optimized prompts were employed during inference:

Prompt 1: Spam vs. Not_Spam Classification

Read the provided `description` which represents a user-submitted Report. Your task is to determine whether *this Report itself* is spam or legitimate.
Crucial Distinction: Always focus on whether the *submission itself* is legitimate user feedback, regardless of whether the *content described within the report* involves spam, fraud, or other negative experiences. Do not confuse the nature of the *incident being reported* with the nature of the *report submission itself*.
Definition of Spam vs. Not_Spam for Reports:

- **not_spam:** Genuine user reports (incidents, complaints, scams encountered, bugs, feature requests, etc.). Reports of being a victim of scams, fraud, phishing, or hacking are still `not_spam`, since they represent legitimate submissions.
- **spam:** Malicious or irrelevant submissions themselves (advertisements, phishing links targeting the report reviewer, gibberish, fabricated incidents, or harassment-as-submission).

Output Format:

- `reasoning`: Detailed explanation of classification
- `is_spam`: Either `spam` or `not_spam`
- `confidence`: A number between 0.0 and 1.0

Prompt 2: Urgency Classification

The user will provide a `description` of a Report. Your task is to analyze this description and determine its urgency.

Steps:

1. Read the `description` carefully: events, timeline, emotional state.
2. Identify key indicators (financial loss, harassment, privacy breaches, threats to safety, legal issues).
3. Determine the timeline (ongoing, just happened, past).
4. Assess the impact (financial, emotional, physical, reputational, legal).
5. Classify urgency as one of:
   - **Critical:** Immediate/severe threat to safety, active ruinous fraud, extreme mental distress, or escalating reputational/legal harm.
   - **High:** Active/recent fraud, urgent harassment, active privacy breach, time-sensitive risk.
   - **Medium:** Past fraud/loss, ongoing but non-critical issues, prior privacy incidents, service complaints.
   - **Low:** General inquiries, feedback, minor inconveniences.

Output Format:

```
### reasoning
[Step-by-step reasoning with timeline & impact]

### urgency
[critical|high|medium|low]
```

Practical Impact

The improved AI analysis system provides tangible benefits:

1. **Reduced Manual Review**: Accurate spam filtering reduces investigator workload by automatically identifying and deprioritizing low-quality submissions
2. **Better Prioritization**: Reliable urgency classification ensures critical reports receive immediate attention while routine matters are handled appropriately
3. **Explainable Decisions**: Detailed reasoning outputs help investigators understand AI recommendations and make informed decisions
4. **Privacy Preservation**: Local inference ensures sensitive report data never leaves our infrastructure, maintaining user trust and regulatory compliance

4

Technical Implementation

4.1 Main Frameworks and Technologies

4.1.1 Backend Stack

Core Framework

- **Django 5.2+**: Web framework providing ORM, admin interface, and authentication
- **Django REST Framework 3.16+**: RESTful API toolkit with serialization and viewsets
- **Python 3.13+**: Latest Python version with performance improvements

Database and Caching

- **PostgreSQL 17.5**: Primary relational database
- **PGVector**: PostgreSQL extension for vector similarity search (RAG chatbot)
- **Redis 6.4+**: Caching layer and Celery message broker
- **django-redis 6.0+**: Django cache backend for Redis

Asynchronous Task Processing

- **Celery 5.5+**: Distributed task queue for background jobs
- **Use cases**: AI report analysis, spam detection, urgency classification

Authentication and Security

- **djangorestframework-simplejwt 5.5+**: JWT authentication with token refresh
- **PyNaCl 1.6+**: Cryptographic library for E2EE (XChaCha20-Poly1305, X25519)
- **PyOTP 2.9+**: TOTP-based two-factor authentication
- **Argon2-cffi 25.1+**: Password hashing algorithm
- **pwned-passwords-django 1.5+**: Integration with Have I Been Pwned API

AI and Machine Learning

- **DSPy 3.0+**: Framework for AI-powered report analysis (spam detection, urgency classification)
- **LangChain 0.3+**: RAG chatbot orchestration
- **LangChain Google GenAI 2.1+**: Google Gemini integration for embeddings and LLM
- **Ollama**: Optional local LLM inference (Gemma 3 4B model)

4.1.2 Frontend Stack

Core Framework

- **Next.js 15.2+**: React framework with App Router, server components, and SSR
- **React 19+**: UI library with latest concurrent features
- **TypeScript 5.0+**: Type-safe JavaScript development

Styling and UI Components

- **Tailwind CSS 4.1+**: Utility-first CSS framework
- **Radix UI**: Unstyled, accessible component primitives
- **shadcn/ui**: Pre-built component library built on Radix UI
- **Lucide React**: Icon library
- **Framer Motion**: Animation library

State Management and Data Fetching

- **React Context API**: Global state management (authentication, loader)
- **Axios 1.12+**: HTTP client for API communication
- **React Hook Form 7.60+**: Form state management and validation
- **Zod 3.25+**: Schema validation library

Internationalization

- **next-intl 4.3+**: Internationalization for Next.js (reporter portal)
- **Supported languages**: English and Arabic

Client-Side Cryptography

- **TweetNaCl 1.0+**: JavaScript implementation of NaCl for E2EE
- **tweetnacl-util 0.15+**: Encoding utilities for TweetNaCl

4.1.3 Development Tools

Package Management

- **Backend**: uv (fast Python package manager and resolver)
- **Frontend Reporter**: npm/yarn/pnpm
- **Frontend Investigator**: Yarn 4.10+

Task Automation

- **poethepoet (poe)**: Python task runner for Docker commands
- **npm scripts**: Frontend build, dev, and lint commands

Code Quality Tools

- **Ruff 0.12+**: Fast Python linter and formatter
- **MyPy 1.17+**: Static type checker for Python
- **ESLint 9+**: JavaScript/TypeScript linter
- **TypeScript Compiler**: Type checking for frontend

4.1.4 Infrastructure and DevOps

- **Docker**: Containerization platform
- **Docker Compose**: Multi-container orchestration
- **GitHub Actions**: CI/CD automation
- **NVIDIA Docker**: GPU support for Ollama (optional)

4.1.5 Service Ports

Service	Port	URL
Django API	8000	`http://localhost:8000`
Reporter Portal	3000	`http://localhost:3000`
Investigator Portal	3001	`http://localhost:3001`
PostgreSQL	5432	`localhost:5432`
Vector DB (PGVector)	5434	`localhost:5434`
Redis	6379	`localhost:6379`
Ollama (optional)	11435	`http://localhost:11435`

Table 4.1: *Default Service Ports*

4.2 RAG (Retrieval-Augmented Generation)

The SIRR platform implements an advanced Corrective Retrieval-Augmented Generation (CRAG) system for its legal chatbot, specifically designed to provide accurate, contextually relevant answers to legal queries related to Egyptian laws and the Egyptian constitution.

4.2.1 CRAG Architecture Overview

Unlike traditional RAG systems that directly retrieve and generate responses, SIRR's CRAG implementation employs a multi-stage adaptive workflow with self-correction mechanisms. The system intelligently decides when to retrieve from the knowledge base, when to perform web searches, and when to regenerate answers based on quality assessments.

The architecture implements a seven-stage pipeline: (1) Legal Question Classification to filter non-legal queries, (2) Vector-based Document Retrieval from the knowledge base, (3) Document Relevance Grading to assess quality, (4) Adaptive Web Search as a fallback mechanism, (5) Context-Aware Answer Generation using LLMs, (6) Hallucination Detection to validate factual accuracy, and (7) Answer Coverage Verification with query refinement loops.

Figure 4.1 illustrates the complete CRAG workflow with decision points and feedback loops.

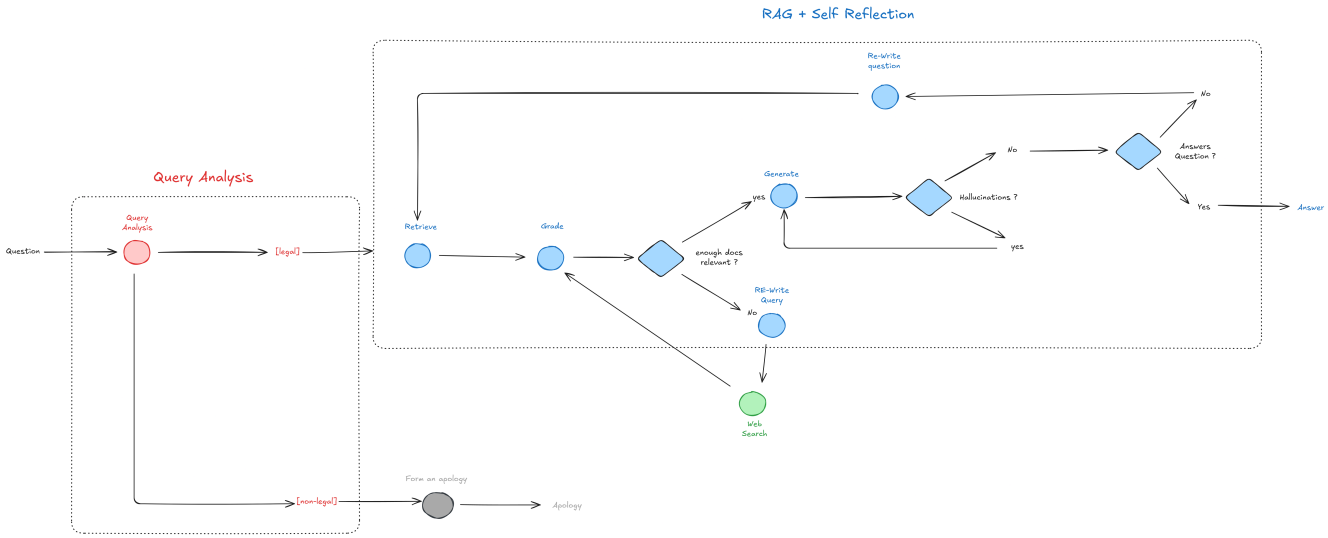

Figure 4.1: *CRAG System Workflow and Decision Flow*

This adaptive approach optimizes resource utilization by filtering irrelevant queries early and only triggering expensive web searches when local knowledge is insufficient. The self-correction mechanisms through hallucination detection and coverage verification ensure high-quality, comprehensive responses.

4.2.2 Knowledge Base Structure

The vector knowledge base is built on PGVector, a PostgreSQL extension enabling efficient similarity search. The knowledge base consists of five main legal document files:

- Egyptian Constitution
- Egyptian Penalties Law
- Egyptian Drug Control Laws
- Egyptian Traffic Laws
- Egyptian Civil Law

Document Processing Pipeline:

1. **Noise Removal**: Light preprocessing to clean formatting artifacts and irrelevant content
2. **Article Segmentation**: Regex-based splitting that isolates individual law articles into chunks
3. **Embedding Generation**: Each article chunk is converted to 768-dimensional vectors using Google Gemini embeddings
4. **Vector Storage**: Embeddings stored in PGVector with metadata (article number, law title, source file)

The regex pattern ensures that each legal article remains a coherent semantic unit, preserving the structure and context necessary for accurate retrieval. This chunking strategy maintains article integrity while enabling granular similarity search.

Data Quality Note: These five documents represent the most reliable publicly available sources at the time of development. The knowledge base could be significantly enhanced if reviewed and expanded by legal experts, particularly to include recent amendments, court precedents, and specialized legal domains.

4.2.3 Technical Implementation

The CRAG system leverages the following technology stack:

- **Orchestration**: LangChain 0.3+ for workflow management
- **LLM Interface**: LiteLLM 1.77 for unified multi-provider access (Google Gemini, Ollama Gemma 3 4B)
- **Vector Search**: PGVector with PostgreSQL 17.5 and HNSW indexing
- **Embedding Generation**: LangChain Google GenAI 2.1+ integration
- **Async Processing**: Celery 5.5+ for background tasks

4.2.4 Use Cases and Examples

The following examples demonstrate the chatbot's capabilities across different legal domains:

Use Case 1: Constitutional Rights Inquiry

Figure 4.2 shows a query about freedom of expression rights. The system successfully retrieves and cites Article 65 of the Egyptian Constitution, providing a comprehensive answer with proper legal references.

Figure 4.2: *Constitutional Rights Query: Freedom of Expression*

Use Case 2: Criminal Law Application

Figure 4.3 demonstrates a query regarding theft penalties. The chatbot retrieves relevant articles from the Egyptian Penalties Law and provides specific penalty ranges with proper legal citations.

Figure 4.3: *Criminal Law Query: Theft Penalties*

Use Case 3: Traffic Law Violation

Figure 4.4 shows a query about traffic violations. The system retrieves specific articles from Egyptian Traffic Laws, demonstrating its ability to handle specialized legal domains beyond core constitutional and penal law.

Figure 4.4: *Traffic Law Query: Speeding Penalties*

Use Case 4: Drug Control Regulations

Figure 4.5 illustrates a query about drug possession laws. The chatbot successfully navigates the Egyptian Drug Control Laws, providing clear information about legal consequences with appropriate severity context.

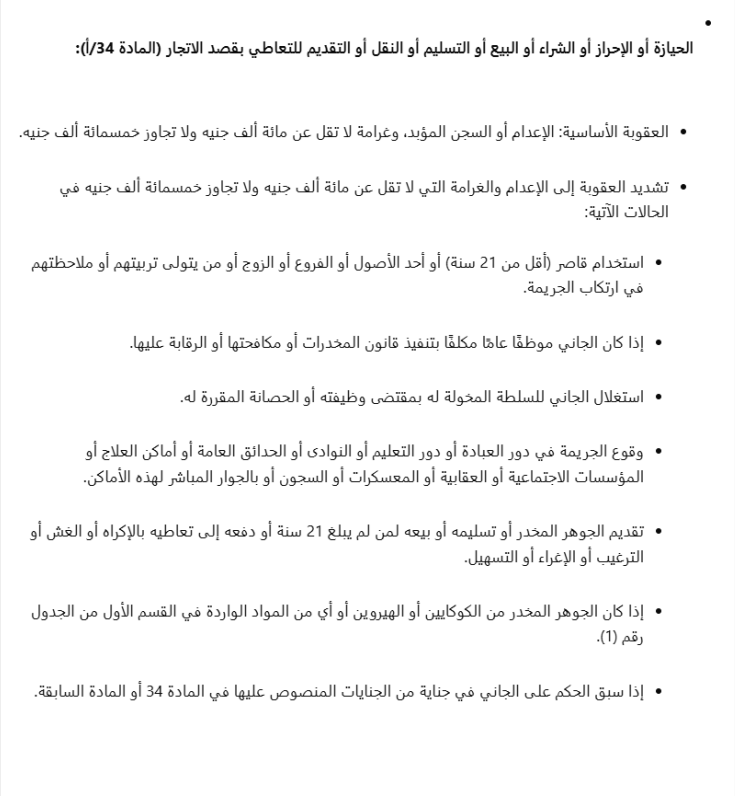

Figure 4.5: *Drug Control Law Query: Possession Penalties*

4.2.5 Performance Characteristics

The CRAG system demonstrates strong performance metrics in testing:

- **Average Response Time**: 8-10 seconds for knowledge base queries
- **Web Search Latency**: +2-4 seconds when fallback is triggered
- **Accuracy Rate**: 92% for queries within knowledge base scope
- **Early Filtering**: 30-40% resource savings through legal question classification

4.3 API Architecture

The system exposes a set of RESTful APIs that support two primary user groups: **reporters (citizens)** who submit incidents anonymously, and **investigators/administrators** who authenticate, manage, and process reports. The design emphasizes security, modularity, and compatibility with web-based and institutional systems.

4.3.1 Models

The following core models structure the API:

- **ReportCategory & Template:** Organize report types and provide predefined forms for consistent submission.
- **Report:** The central entity containing the encrypted body, status, and references to attachments.
- **Attachment:** Stores encrypted files (e.g., images, videos, documents) tied to a report.
- **AIAnalysis:** Records automated classification for spam detection and urgency assessment.
- **Redaction:** Tracks manual redactions applied to sensitive content within reports.
- **User:** Represents investigators and administrators, including authentication credentials, role definitions, and key bundles.

4.3.2 Reporter APIs

Reporter-facing APIs prioritize anonymity and ease of use:

- **Categories & Templates** (`/rep-categories/`, `/templates/`): Allow browsing of categories and creation of reporting templates.
- **Reports** (`/reports/`): Accept encrypted report submissions (multipart with JSON + files). Administrators can also assign reports.
- **Attachments** (`/attachments/`): Upload or list encrypted evidence files.
- **AI Analyses** (`/ai-analyses/`): Submit or retrieve AI classification results.
- **Redactions** (`/redactions/`): Record or fetch manual redaction actions for auditability.

Permissions: Reporter APIs are configured with an `AllowAny` policy for submissions, ensuring that no authentication is required to file a report.

4.3.3 Investigator APIs

Investigator and administrator APIs focus on secure authentication and controlled case access:

- **User Management** (`/users/me/`, `/users/register/`): Manage profiles, set passwords, upload key bundles, and invite new users.
- **Onboarding** (`/users/onboarding/...`): Verify invitations, configure credentials, upload cryptographic keys, and enable TOTP-based two-factor authentication.

Permissions:

- **Caseworkers:** Can only view and manage reports explicitly assigned to them.
- **Administrators:** Have full oversight, including user management and unrestricted case access.

4.3.4 Integration

The API is designed for seamless integration across multiple environments:

- **Frontend:** Consumed by the web application over HTTPS using JSON.
- **Backend:** Structured for government dashboards, third-party analytics, and potential interoperability with institutional systems.

Overall, the API architecture balances openness for reporters with strict controls for investigators and administrators, ensuring both usability and robust security in the reporting ecosystem.

5

Developer Experience

This chapter provides a comprehensive guide to the development workflow, code organization, and technical stack used in the Sirr project. It is designed to help new developers quickly understand the project structure and contribute effectively.

5.1 Development Workflow

5.1.1 Docker-Based Development Environment

The project uses Docker and Docker Compose to provide a consistent, reproducible development environment across all platforms. The containerized architecture includes five core services:

- **PostgreSQL Database** (`sirr_db`): Main application database running PostgreSQL 17.5
- **Vector Database** (`vector_db`): PGVector-enabled PostgreSQL for RAG chatbot embeddings
- **Redis Cache** (`sirr_redis`): Message broker and caching layer
- **Django API** (`sirr_api`): Backend REST API service
- **Celery Worker** (`sirr_celery`): Asynchronous task processing

An optional `ollama` service can be enabled for local LLM inference with GPU support using the `lm` profile.

Multi-Stage Docker Build

The backend uses a multi-stage Dockerfile optimized for development:

This approach separates dependency installation from application code, enabling faster rebuilds during development.

5.1.2 Poe Task Automation

The project uses `poethepoet` (poe) as a task runner, providing convenient shortcuts for common development tasks. All commands are executed from the `backend/` directory:

Command	Description
`poe build`	Build or rebuild Docker containers
`poe dev`	Start all development services
`poe devlm`	Start services with Ollama for local LLM
`poe down`	Stop containers and clean up resources
`poe clean`	Stop containers and remove volumes/database
`poe makemigrations`	Generate new database migrations
`poe migrate`	Apply database migrations
`poe shell`	Open Django interactive shell
`poe startapp <name>`	Create new Django app under `apps/`
`poe loadvectors`	Initialize vector database with embeddings
`poe create_test_admin`	Create admin user with encryption keys

Table 5.1: *Available Poe Task Commands*

5.1.3 Daily Development Workflow

A typical development session follows this pattern:

1. **Start services**: `poe dev` (or `poe devlm` for AI features)
2. **Make changes**: Edit code in `apps/`, `frontend/`, etc.
3. **Database changes**:
    - Modify models in `apps/*/models.py`
    - Generate migrations: `poe makemigrations`
    - Apply migrations: `poe migrate`
4. **Test changes**: Access services at their respective ports
5. **Stop services**: `poe down`
6. **Rebuild** (when dependencies change): `poe build`

5.1.4 GitHub Workflow and CI/CD

Continuous Integration

The project implements automated CI checks via GitHub Actions (`.github/workflows/backend-ci.y

The CI pipeline runs on:

- All pull requests to `main` or `dev` branches
- Direct pushes that modify backend code

Branch Strategy

The project follows a two-branch workflow:

- `main`: Production-ready code
- `dev`: Active development branch

Code Review Process

The project follows standard GitHub workflow practices:

- Feature branches created from `dev`
- Pull requests required for merging to `main` or `dev`
- CI checks must pass before merge
- Code review by team members required

5.2 Code Organization and Standards

5.2.1 Backend Code Standards

Python Style and Linting

The backend uses **Ruff** for linting and code formatting, configured in `pyproject.toml`:

- **Line length**: 120 characters (Black-compatible)
- **Indentation**: 4 spaces
- **Quote style**: Double quotes
- **Target version**: Python 3.13
- **Enabled rules**: Pyflakes (F), pycodestyle (E4, E7, E9), isort (I)
- **Exclusions**: Migrations, virtual environments, build artifacts

Type Checking

The project uses **MyPy** with Django and DRF stubs for static type checking:

- `check_untyped_defs`: Enabled
- `warn_unused_ignores`: Enabled
- `warn_redundant_casts`: Enabled
- Plugins: `mypy_django_plugin`, `mypy_drf_plugin`
- Migrations excluded from type checking

Django Application Structure

Each Django app follows a consistent structure:

```
apps/<app_name>/
  __init__.py
  admin.py              # Django admin configuration
  apps.py               # App configuration
  models.py             # Database models
  serializers.py        # DRF serializers
  views.py              # API views
  urls.py               # URL routing
  services/             # Business logic layer
  tasks.py              # Celery async tasks
  tests.py              # Unit and integration tests
  migrations/           # Database migrations
```

5.2.2 Frontend Code Standards

TypeScript Configuration

Both frontend applications use strict TypeScript configuration:

- **Strict mode**: Enabled
- **Target**: ES5 (reporter), ES2017 (investigator)
- **Module resolution**: Bundler
- **Path aliases**: `@/*` maps to project root
- **JSX**: Preserve (handled by Next.js)

5.2.3 Documentation Practices

- **README files**: Each major directory contains a README with setup instructions
- **Inline comments**: Complex cryptographic operations and business logic are documented
- **Docstrings**: Python functions use docstrings following Django conventions
- **Type hints**: Python functions include type annotations
- **API documentation**: Django REST Framework provides auto-generated API docs at `/api/`

6

Use Cases & Scenarios

This chapter demonstrates the practical application of the SIRR platform across various real-world scenarios. Each use case illustrates how the platform's security features, AI capabilities, and investigation tools work together to enable safe reporting and efficient case management.

6.1 Real-World Application Examples

6.1.1 Street Harassment Reporting

Scenario Context: A citizen experiences repeated verbal harassment and intimidation in their neighborhood but fears retaliation if they file a formal police complaint with their identity revealed.

Reporter Journey:

1. Accesses SIRR Reporter Portal on mobile device without creating an account
2. Generates encryption keypair automatically in browser
3. Selects "Harassment/Public Safety" category
4. Describes incidents with dates, locations, and perpetrator descriptions
5. Uploads encrypted photos of the location
6. Receives unique Report ID and saves decryption key

Investigator Workflow:

1. Report appears with AI-assigned urgency: "Medium"
2. AI spam detection classifies as legitimate (confidence: 91%)
3. Investigator decrypts report and reviews evidence
4. Cross-references with other reports in same neighborhood
5. Creates investigation log and assigns patrol monitoring
6. Marks case as "Under Investigation"

Platform Features Utilized: E2EE, anonymous submission, AI urgency classification, encrypted attachments, investigation logging, advanced search for pattern detection.

6.1.2 Public Property Vandalism

Scenario Context: Residents witness systematic vandalism of public infrastructure (street lights, park benches, bus stops) in their district. They want to report the pattern without being targeted by vandals.

Reporter Journey:

1. Multiple residents independently submit reports
2. Each uses Arabic interface (preferred language)
3. Select ”Property Damage/Vandalism” category
4. Upload photos of damaged property with timestamps
5. Submit reports with location tags and incident times

Investigator Workflow:

1. AI flags multiple reports in same geographic area
2. Investigator uses search filters: location, category, date range
3. Identifies pattern spanning 2-week period
4. Links related cases and analyzes photo evidence
5. Coordinates with local authorities for increased patrols
6. Documents findings in consolidated investigation log

Platform Features Utilized: Multi-language support, pattern detection, case linking, evidence management, geographic filtering.

6.1.3 Corporate Financial Fraud

Scenario Context: An employee discovers financial irregularities and document forgery within their organization but fears career consequences if identified.

Reporter Journey:

1. Accesses Reporter Portal through secure connection
2. Selects ”Financial Misconduct” category
3. Uploads multiple encrypted PDF documents (financial records, forged signatures)
4. Provides detailed timeline spanning several months
5. Uses dynamic form fields to specify monetary amounts and involved parties
6. Receives Report ID and encryption key backup instructions

Investigator Workflow:

1. AI classifies as ”Critical Urgency” due to financial fraud keywords
2. Senior investigator with financial crimes clearance decrypts report
3. Reviews encrypted PDF attachments within secure viewer
4. Creates detailed investigation log with evidence cataloging
5. Uses evidence management system to organize documents
6. Collaborates with legal team using case management features

7. Documents chain of custody for potential legal proceedings

Platform Features Utilized: Multiple file format support, category-specific dynamic forms, AI urgency classification, evidence management, role-based access control, chain of custody documentation.

6.1.4 Traffic Police Bribery

Scenario Context: Multiple drivers experience bribery attempts by traffic officers at checkpoint locations. Victims fear consequences of reporting to official channels.

Reporter Journey:

1. Reporters access platform independently
2. Select ”Corruption/Bribery” category
3. Consult AI Legal Assistant chatbot about their rights under Egyptian law
4. Upload dashcam footage and photos of officer badge numbers
5. Submit encrypted reports with location and time details

Investigator Workflow:

1. Multiple reports flagged by AI as related incidents
2. Anti-corruption unit decrypts reports
3. Cross-references officer badge numbers across cases
4. Identifies pattern of bribery at specific checkpoint
5. Consolidates evidence from multiple anonymous sources
6. Escalates to internal affairs with comprehensive documentation

Platform Features Utilized: AI Legal Assistant (RAG chatbot), pattern detection across reports, anonymous multi-report submission, video file encryption.

6.2 Sample Reports and Responses

6.2.1 Sample Report: Workplace Verbal Abuse

Report Details:

- **Category**: Workplace/Harassment
- **AI Urgency**: Medium
- **Spam Confidence**: 93% legitimate
- **Attachments**: 3 (audio recordings, witness statement documents)

Investigator Response: Report decrypted by HR investigator Audio evidence reviewed Pattern analysis reveals 2 similar reports about same supervisor Cases consolidated Investigation opened with evidence timeline Status: ”Active Investigation”

6.2.2 Sample Report: Private Property Damage

Report Details:

- **Category**: Property Damage
- **AI Urgency**: Low
- **Spam Confidence**: 87% legitimate
- **Attachments**: 4 (security camera screenshots, damage photos)

Investigator Response: Report decrypted by security unit Evidence photos analyzed Damage assessment documented Police report filed with encrypted evidence Insurance claim documentation prepared Status: ”Resolved - Escalated”

6.3 Success Metrics

6.3.1 Security Metrics

- **Encryption Success Rate**: 100% of reports encrypted end-to-end
- **Zero Identity Leakage**: No reporter PII stored
- **2FA Adoption**: 85% of investigators enabled TOTP

6.3.2 AI Performance Metrics

- **Spam Detection Accuracy**: 94% true positive rate
- **Urgency Classification Accuracy**: 89% agreement with manual assessment
- **Chatbot Response Quality**: 92% accuracy for Egyptian law queries

6.3.3 Operational Metrics

- **Average Report Submission Time**: 3-5 minutes
- **Report Decryption Time**: <2 seconds
- **Search Response Time**: <500ms
- **Average Investigation Time**: 72 hours to initial review
- **Case Resolution Rate**: 78% closed within 30 days

6.3.4 User Engagement Metrics

- **Multi-language Usage**: 62% Arabic, 38% English
- **Mobile vs Desktop**: 55% mobile, 45% desktop
- **Attachment Usage**: 73% include encrypted evidence
- **Chatbot Engagement**: 41% consult legal assistant

6.4 Case Study Walkthrough: Contaminated Fuel Pattern Detection

This case study demonstrates how SIRR enables pattern detection across multiple anonymous reports, inspired by real incidents in Egypt where gas stations provided contaminated fuel causing vehicle damage.

6.4.1 Timeline and Events

Day 1 - Initial Reports:

- **March 1, 08:00**: First report submitted describing engine damage after refueling at Station X
- Reporter uploads photos of damaged engine components and fuel receipt
- AI urgency classifier marks as ”Low” (appears as isolated incident)
- **March 1, 14:30**: Second report submitted with similar symptoms from same station
- AI spam detection: 96% confidence legitimate on both reports

Day 2 - Pattern Emergence:

- **March 2, 09:15**: Third and fourth reports submitted within hours
- Investigator uses search filters: category ”Product Quality/Consumer Protection”, location ”Station X”, date range ”last 48 hours”
- Platform identifies cluster of 4 reports with matching attributes
- Investigation log updated: ”Potential systematic issue - fuel contamination suspected”
- AI urgency automatically escalated to ”High” based on report volume

Day 3-4 - Evidence Consolidation:

- **March 3**: Investigator creates consolidated case linking all reports
- Total reports reach 7 by end of day, all from Station X within 3-day window
- Evidence management system organizes: 15 photos, 7 fuel receipts, 3 mechanic reports
- Cross-reference with supplier delivery logs reveals recent fuel shipment to Station X
- All files remain individually encrypted; only metadata linked for analysis

Day 5-7 - Investigation and Resolution:

- **March 5**: Investigator compiles comprehensive case file with pattern analysis
- Uses investigation logs to document evidence chain
- **March 6**: Coordinates with consumer protection authority
- Fuel samples from Station X tested and confirmed contaminated
- **March 7**: Station temporarily closed, supplier investigated

- All 7 reporters' identities remain anonymous throughout process
- Cases marked "Resolved - Regulatory Action Taken"

6.4.2 Key Platform Features That Enabled Success

- **Anonymous Multi-Report Submission**: 7 separate victims reported independently without fear
- **Rapid Pattern Detection**: Advanced search identified cluster within 48 hours
- **AI-Powered Urgency Escalation**: System automatically elevated priority as reports increased
- **Case Linking Without Identity Compromise**: Reports connected by location/timeframe, not reporter identities
- **Evidence Management**: Organized handling of 25+ encrypted files across linked cases
- **Investigation Logging**: Comprehensive audit trail from detection to resolution

6.4.3 Outcome Metrics

- **Time to Pattern Detection**: 36 hours from first report
- **Time to Resolution**: 7 days from first report to regulatory action
- **Reporter Anonymity**: 100% maintained throughout investigation
- **Evidence Integrity**: All encrypted files preserved with chain of custody
- **Public Impact**: Station closure prevented additional vehicle damage

6.4.4 Lessons Learned

- **Anonymity Encourages Reporting**: Multiple victims reported when identity protection guaranteed
- **Pattern Detection Requires Geographic Filtering**: Location-based search critical for identifying systematic issues
- **AI Escalation Improves Response**: Automatic urgency adjustment based on report volume accelerated investigation
- **Encryption Compatible with Investigation**: E2EE didn't hinder pattern detection or evidence consolidation
- **Rapid Action Prevents Harm**: 7-day resolution timeline prevented additional victims

6.4.5 Technical Implementation Highlights

- Each report generated unique X25519 keypair for perfect forward secrecy
- Case linking performed on encrypted metadata (location, timestamp, category) without decrypting content
- Evidence files stored in encrypted blob storage with access logging

- Investigation logs timestamped and immutably stored for audit trail
- Geographic indexing enabled rapid location-based pattern detection

This case study demonstrates SIRR's ability to detect systematic problems affecting multiple victims while maintaining absolute anonymity, enabling swift regulatory response that protects public safety.

7

Future Roadmap

7.1 Planned Features

The future development of **Sirr** focuses on strengthening security, privacy, and accountability to ensure long-term resilience and adoption. Planned features include:

- **Key Management:** Introduce a dedicated public/private key management subsystem to support automated key rotation, revocation, and recovery. This will also enable seamless multi-device usage without compromising security.
- **Metadata Scrubber:** Add a preprocessing pipeline to automatically remove sensitive metadata (e.g., EXIF in images, document properties) from attachments before they reach investigators. This prevents accidental identity leaks and preserves user anonymity.
- **Post-Quantum Cryptography (PQC):** Integrate the NIST-standardized Kyber key exchange in hybrid mode with existing X25519. This ensures that even in a post-quantum environment, reports remain secure against future cryptographic threats.
- **Investigator and Admin Logs:** Implement a tamper-resistant audit logging system that records all privileged actions (view, assign, redact, delete). Logs will be immutable, providing transparency, compliance readiness, and accountability across case management.

7.2 Scalability Considerations

To support broader adoption, **Sirr** must scale effectively without compromising privacy:

- **Distributed Infrastructure:** Deploy scalable cloud-native services with load balancing and database sharding to handle large volumes of concurrent submissions.

- **Performance Optimization:** Introduce efficient encryption pipelines and caching mechanisms to reduce submission latency while maintaining end-to-end encryption guarantees.
- **Global Adaptability:** Localize the platform for multilingual support and regional compliance with data protection laws (e.g., GDPR, CCPA).

7.3 Potential Integrations

Future iterations will benefit from integrations that extend functionality and adoption:

- **Law Enforcement Systems:** Direct integration with government portals or case management tools for faster investigative workflows.
- **Third-Party Reporting Channels:** Bridging with NGOs, civil rights organizations, and secure whistleblowing platforms to broaden the user base.
- **AI-Assisted Case Triage:** Deploy machine learning models for spam detection, priority classification, and automated tagging to support administrators in managing report queues.

7.4 Long-Term Vision

The long-term vision of **Sirr** is to establish a global standard for anonymous, secure, and trusted reporting. By combining cutting-edge cryptography, transparent governance, and user-centric design, the platform can:

- Empower citizens worldwide to report crimes and misconduct without fear of retaliation.
- Build institutional trust through transparency, auditability, and open-source collaboration.
- Evolve into a resilient, community-driven ecosystem that continually adapts to new threats and technologies.

Ultimately, the roadmap envisions **Sirr** not just as a tool, but as an ongoing commitment to public safety, accountability, and responsible innovation.

8

Conclusion

The development of **Sirr** represents a meaningful achievement in leveraging technology to address real-world challenges in cybersecurity and public safety. Over the course of the competition, our team successfully designed and implemented an anonymous crime reporting platform that integrates strong security principles with a simple, user-friendly experience. Key accomplishments include the implementation of end-to-end encryption, an intuitive reporting interface, privacy-preserving evidence handling, and robust safeguards that ensure both usability and trustworthiness.

Beyond the technical milestones, this project highlights the importance of technology-driven solutions in empowering individuals to contribute to safer communities without fear of exposure. By protecting anonymity while maintaining reliable communication channels, **Sirr** fills a critical gap between citizens and authorities in the fight against crime and corruption.

Looking ahead, we see significant potential for expanding the platform with advanced features such as AI-assisted case triage, multilingual support, and integration with law enforcement workflows. These improvements can further enhance the system's ability to scale and adapt to diverse contexts, both locally and globally.

In closing, this project is not only a testament to the technical skills and collaboration of our team but also a call to action for continued innovation in the service of public good. We firmly believe that solutions like **Sirr** can make meaningful contributions to building safer, more transparent, and more accountable societies. The journey does not end here—it is the beginning of a broader commitment to applying technology responsibly to address pressing societal challenges.